Advanced Statistical Inference -- Bayesian Linear Regression

In this session, you'll start to implement some basic Bayesian models, starting from the simple Bayesian linear regression that we saw during the lecture.

In [146…

```
import functools
import warnings

import matplotlib
import matplotlib.font_manager
import matplotlib.pyplot as plt
import numpy as np
import scipy.linalg
import scipy.stats
from matplotlib import rc
from mpl_toolkits.mplot3d import Axes3D

colab = "google.colab" in str(get_ipython())
preamble = r"""\renewcommand{\familydefault}{\sfdefault}
            \usepackage{sansmath} \sansmath  \usepackage{amsmath}"""
rc("font", **{"family": "sans-serif", "sans-serif": "DejaVu Sans"})
rc("text", **{"usetex": False, "latex.preamble": preamble})
rc("figure", **{"dpi": 200})
rc(
    "axes",
    **{"spines.right": False, "spines.top": False, "xmargin": 0, "ymargin":
)
```

1. A regression dataset

Create a simple 1D regression dataset using the `make_regression(...)` function and plot it. For the moment, keep the noise variance σ_{n} small. NB. For better reproducibility, please remember to fix the Numpy's random seed. For Jupyter notebooks, this needs to be done at the beginning of all cells.

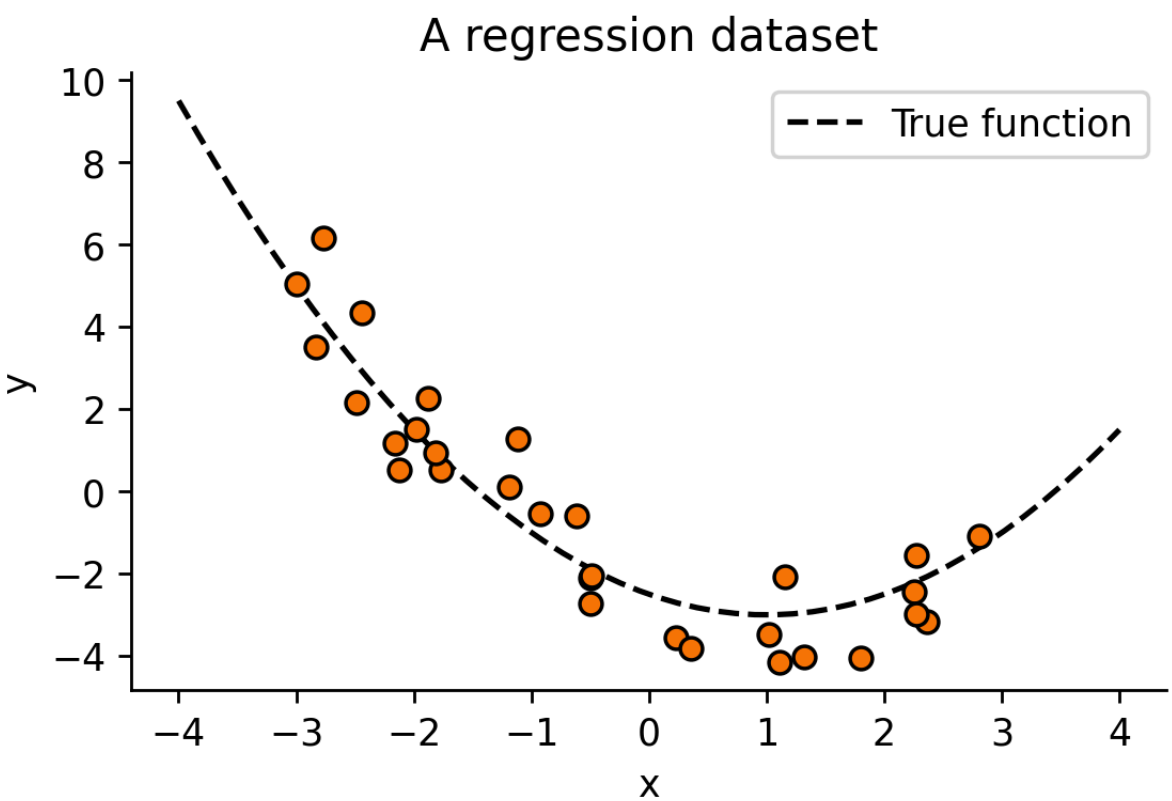
In [147…

```
true_function = lambda x: 0.5 * ((x - 1) ** 2) - 3


def make_regression(n, sn2=0.1):
    x = np.random.uniform(-3, 3, n)
    y = true_function(x) + np.sqrt(sn2) * np.random.randn(*x.shape)
    return x, y
```

In [148…

```
np.random.seed(1)
sn2 = 1.5
x, y = make_regression(30, sn2=sn2)
xp = np.linspace(-4, 4)
```

```
fig, ax = plt.subplots(figsize=[5, 3], dpi=250)
ax.plot(xp, true_function(xp), "--k", zorder=0, label="True function")
ax.scatter(x, y, edgecolor="black", linewidth=1, facecolor="xkcd:orange")
ax.set_title("A regression dataset")
ax.legend()
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.margins(0.05)
plt.show()
```



2. A review on Gaussian likelihood

Let's start from the basics. Remember what a likelihood is. The **likelihood** measures the goodness of fit of a statistical model to samples of data for given values of the unknown model parameters. It's computed from the joint probability distribution, but viewed and used as **function** of the parameters only, thus treating the random variables as fixed at the observed values.

A Gaussian likelihood is defined as

$$p(\mathbf{y}|\mathbf{w}, \mathbf{X}, \sigma_\mathrm{n}) = \prod_{i=1}^{N} p(y_i|\mathbf{w}, \mathbf{x}_i, \sigma_\mathrm{n}) = \prod_{i=1}^{N} \mathcal{N}(y_i|\tilde{y}_i, \sigma_\mathrm{n}) \tag{1}$$

where, for linear regression, $\tilde{y}_i = \mathbf{w}^\top \mathbf{x}_i$. For numerical stability, instead of using the vanilla likelihood, we will use the **log-likelihood**.

$$\log p(\mathbf{y}|\mathbf{w}, \mathbf{X}, \sigma_\mathrm{n}) = \sum_{i=1}^{N} \log \mathcal{N}(y_i|\tilde{y}_i, \sigma_\mathrm{n}) \tag{2}$$

Exercise: Write a function to compute the log-density of a normal distribution at position x, given μ and σ^2.

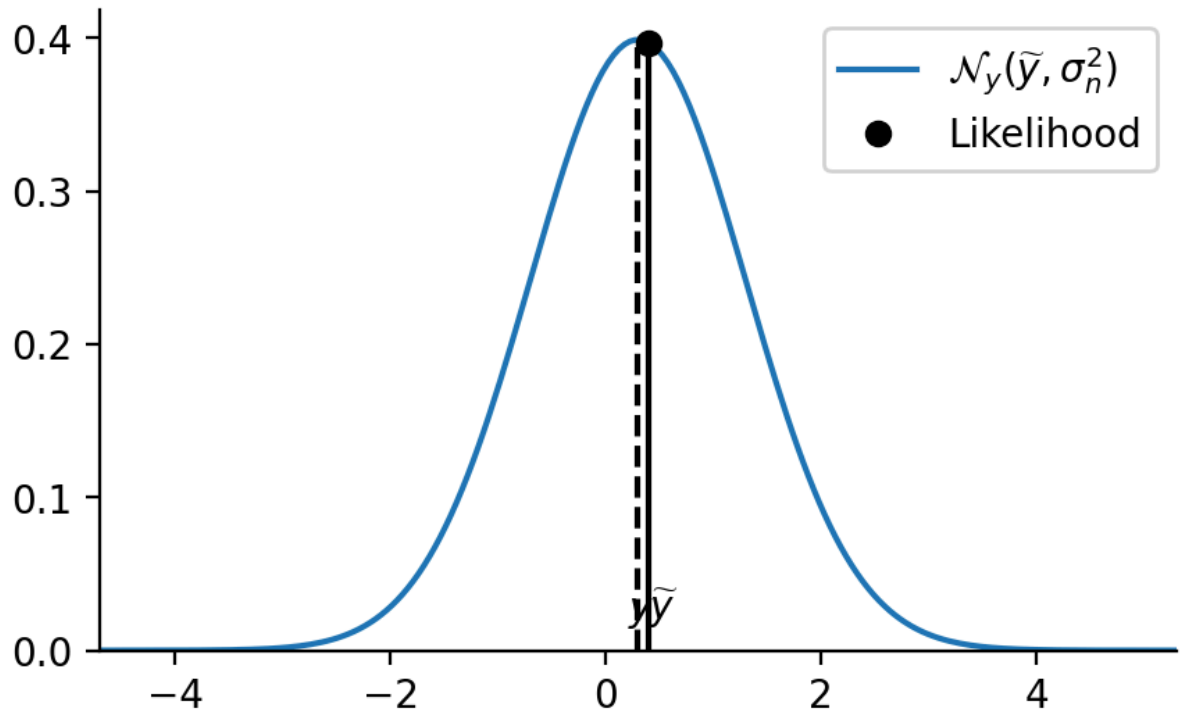
In [149…

```
def lognormal(x, mu, var):
    # @@ COMPLETE @@ #
    value = -0.5 * np.log(2 * np.pi * var) - 0.5 * ((x - mu) ** 2) / var
    return value
```

Exercise: For the moment, assume that for the sample i^{th}, you predict $\tilde{y} = 0.3$ and $\sigma_\mathrm{n} = 1$. You know that $y = 0.4$. Complete the following function `gaussian_loglik(...)`, then compute the (log)likelihood for this sample and show its position on the Gaussian density with a plot.

In [150…

```
def gaussian_loglik(y, y_tilde, sn2):
    # @@ COMPLETE @@ #
    ll = lognormal(y, y_tilde, sn2)
    return ll


def plot_gaussian(mu, var, plot_log=False, **kwargs):
    """A simple util to plot a gaussian pdf"""
    x = np.linspace(mu - 5 * np.sqrt(var), mu + 5 * np.sqrt(var), 200)
    y = lognormal(x, mu, var) if plot_log else np.exp(lognormal(x, mu, var))
    ax = kwargs.pop("ax", plt.gca())
    ax.plot(x, y, **kwargs)
    return ax


# @@ COMPLETE @@ #
y_obs = 0.4
y_tilde = 0.3
sn2 = 1.0
sample_ll = gaussian_loglik(y_obs, y_tilde, sn2)


fig, ax = plt.subplots(figsize=[5, 3])
plot_gaussian(y_tilde, sn2, ax=ax, label=r"$\mathcal{N}_y(\widetilde{y}, \si
ax.vlines(y_obs, 0, np.exp(sample_ll), color="k")
ax.vlines(
    y_tilde, 0, np.exp(gaussian_loglik(y_tilde, y_tilde, sn2)), ls="--", col
)
ax.plot(y_obs, np.exp(sample_ll), "ok", label=r"Likelihood")
ax.text(y_tilde + 0.1, 0.02, r"$\widetilde{y}$")
ax.text(y_obs - 0.2, 0.02, r"$y$")
ax.set_ylim(0)
ax.legend()
plt.show()
```



3. Bayesian Linear regression

In this section, you'll start to implement the Bayesian linear regression model. Let's start by creating the **design matrix** $\mathbf{X}$.

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^1 & \dots & x_1^K \\ 1 & x_2^1 & \dots & x_2^K \\ \vdots & \vdots & & \vdots \\ 1 & x_N^1 & \dots & x_N^K \end{bmatrix}$$

Exercise: Complete the following function `build_features(...)` to build $\mathbf{X}$. This can be done in many ways. One of them is using a double list comprehension (one index for the row and one for the column), while another one is using the numpy `column_stack()` function (highly suggested). In any case, inspect $\mathbf{X}$ to make sure it looks OK (show the first entries). To fit higher order polynomials, we need to add extra columns to $\mathbf{X}$, therefore build it with K as a parameter.

In [151…
```
def build_features(x, K):
    # @@ COMPLETE @@ #
    X = np.column_stack([x**k for k in range(K + 1)])
    return X
```

In [152…
```
build_features(x, 2)
```

Out[152…

```
array([[ 1.        , -0.49786797,  0.24787252],
       [ 1.        ,  1.32194696,  1.74754377],
       [ 1.        , -2.99931375,  8.99588298],
       [ 1.        , -1.18600456,  1.40660683],
       [ 1.        , -2.11946466,  4.49213042],
       [ 1.        , -2.44596843,  5.98276157],
       [ 1.        , -1.88243873,  3.54357558],
       [ 1.        , -0.92663564,  0.85865361],
       [ 1.        , -0.61939515,  0.38365036],
       [ 1.        ,  0.2329004 ,  0.0542426 ],
       [ 1.        , -0.48483291,  0.23506295],
       [ 1.        ,  1.111317  ,  1.23502548],
       [ 1.        , -1.7732865 ,  3.14454502],
       [ 1.        ,  2.26870462,  5.14702065],
       [ 1.        , -2.83567444,  8.04104953],
       [ 1.        ,  1.02280506,  1.04613019],
       [ 1.        , -0.49617119,  0.24618585],
       [ 1.        ,  0.35213897,  0.12400185],
       [ 1.        , -2.15767837,  4.65557594],
       [ 1.        , -1.81139107,  3.28113759],
       [ 1.        ,  1.80446741,  3.25610264],
       [ 1.        ,  2.80956945,  7.89368052],
       [ 1.        , -1.11945493,  1.25317934],
       [ 1.        ,  1.15393569,  1.33156759],
       [ 1.        ,  2.25833491,  5.10007658],
       [ 1.        ,  2.36763998,  5.60571908],
       [ 1.        , -2.48973473,  6.19877903],
       [ 1.        , -2.7656713 ,  7.64893774],
       [ 1.        , -1.98101748,  3.92443027],
       [ 1.        ,  2.26885502,  5.1477031 ]])
```

From the lecture notes, let's define the prior on the parameters $\mathbf{w}$ as

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{S}) \tag{3}$$

For sake of simplicity, assume the covariance matrix $\mathbf{S}$ to be diagonal $\mathbf{S} = \sigma_{\mathrm{w}}^2 \mathbf{I}$. Remember that the likelihood is defined as $p(\mathbf{y}|\mathbf{w}, \mathbf{X}, \sigma_{\mathrm{n}}) = \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma_{\mathrm{n}}^2 \mathbf{I})$. In this case, the posterior is analitic and follows this form:

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \sigma_{\mathrm{n}}) = \mathcal{N}\left(\frac{1}{\sigma_{\mathrm{n}}^2} \mathbf{\Sigma} \mathbf{X}^\top \mathbf{y}, \mathbf{\Sigma}\right) \tag{4}$$

where $\mathbf{\Sigma}^{-1} = \left(\frac{1}{\sigma_{\mathrm{n}}^2} \mathbf{X}^\top \mathbf{X} + \mathbf{S}^{-1}\right)$.

Question: What is the dimensionality of $\mathbf{\Sigma}$? How much does it cost to compute that inverse? Do you know which algorithm you should use to have numerically stable results? Remember that computing $\mathbf{A}^{-1}\mathbf{z}$ means in practice solving a linear system.

Exercise: Complete the following function to compute the posterior.

Sigma is in the dimension of the number of features, which is $K + 1$ (including the bias term). It costs $O((K+1)^3)$ to compute the inverse, but we can use Cholesky

decomposition to solve the linear system without explicitly computing the inverse, which is more numerically stable. Can use `np.linalg.solve` or `scipy.linalg.cho_solve` for this purpose.

In [153…

```
def compute_posterior(X, y, sw2, sn2):
    # @@ COMPLETE @@ #
    d = X.shape[1]
    A = (X.T @ X) / sn2 + np.eye(d) / sw2
    w_posterior_mean = np.linalg.solve(A, X.T @ y / sn2)
    w_posterior_cov = np.linalg.inv(A)

    return w_posterior_mean, w_posterior_cov
```

Execise: Compute the posterior for the regression dataset. For the moment, place $\sigma_{\mathrm{w}}^2 = 1$ and start with polynomial of order 1. Finally, print the posterior mean and covariance. Comment the results.

In [154…

```
sw2 = 1
K = 1

# @@ COMPLETE @@ #
X = build_features(x, K)
w_posterior_mean, w_posterior_cov = compute_posterior(X, y, sw2, sn2)


print(w_posterior_mean)
print(w_posterior_cov)
```

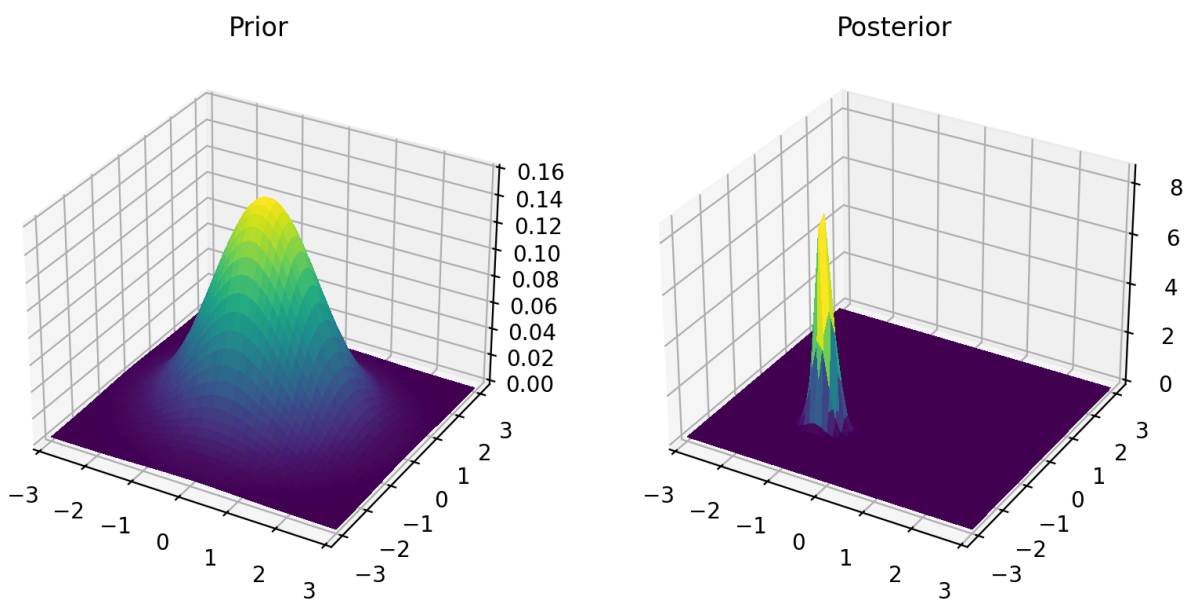
```
[-0.95517429 -1.27532442]
[[0.03367896 0.00379089]
 [0.00379089 0.01011392]]
```

The results should be close to the ones obtained with the maximum likelihood estimation, but the posterior covariance should give you an idea of how uncertain the model is about the parameters.

In [155…

```
x_ = np.linspace(-3, 3, 50)
y_ = np.linspace(-3, 3, 50)
X_, Y_ = np.meshgrid(x_, y_)
pos = np.empty(X_.shape + (2,))
pos[:, :, 0] = X_
pos[:, :, 1] = Y_


fig = plt.figure(figsize=plt.figaspect(0.5))
ax = fig.add_subplot(1, 2, 1, projection="3d")

rv = scipy.stats.multivariate_normal(np.zeros(2), sw2 * np.eye(2))
plot_config = dict(
    cmap="viridis", linewidth=0, antialiased=False, ccount=500, rcount=500
)
ax.plot_surface(X_, Y_, rv.pdf(pos), **plot_config)
ax.set_xlabel("")
```

```
ax.set_ylabel("")
ax.set_zlabel("")
ax.set_title("Prior")

ax = fig.add_subplot(1, 2, 2, projection="3d")
rv = scipy.stats.multivariate_normal(w_posterior_mean, w_posterior_cov)
ax.plot_surface(X_, Y_, rv.pdf(pos), **plot_config)
ax.set_xlabel("")
ax.set_ylabel("")
ax.set_zlabel("")

ax.set_title("Posterior")
plt.show()
```



4. Predictions

Now it's time to make predictions. All our motivation for being Bayesian was to be able to average predictions at $\mathbf{x}_{\text{new}}$, for all possible $\mathbf{w}$. This is possible by computing the following expectation:

$$\mathbf{E}_{p(\mathbf{w}|\mathbf{X},\mathbf{y},\sigma_{\text{n}})}\mathcal{N}(\mathbf{w}^\top\mathbf{x}_{\text{new}},\sigma_{\text{n}}^2) = \int \mathcal{N}(\mathbf{w}^\top\mathbf{x}_{\text{new}},\sigma_{\text{n}}^2)p(\mathbf{w}|\mathbf{X},\mathbf{y},\sigma_{\text{n}})\mathrm{d}\mathbf{w} \tag{5}$$

Question: Prove that $\mathbf{E}_{p(\mathbf{w}|\mathbf{X},\mathbf{y},\sigma_{\text{n}})}\mathcal{N}(\mathbf{w}^\top\mathbf{x}_{\text{new}},\sigma_{\text{n}}^2) = \mathcal{N}(\mathbf{x}_{\text{new}}^\top\mu, \sigma_{\text{n}}^2 + \mathbf{x}_{\text{new}}^\top\mathbf{\Sigma}\mathbf{x}_{\text{new}})$, where μ and $\mathbf{\Sigma}$ are the posterior mean and covariance.

Exercise: Write a function to compute the predictive distribution. Usually we want to do this for multiple points, hence the for-loop.

In [156…

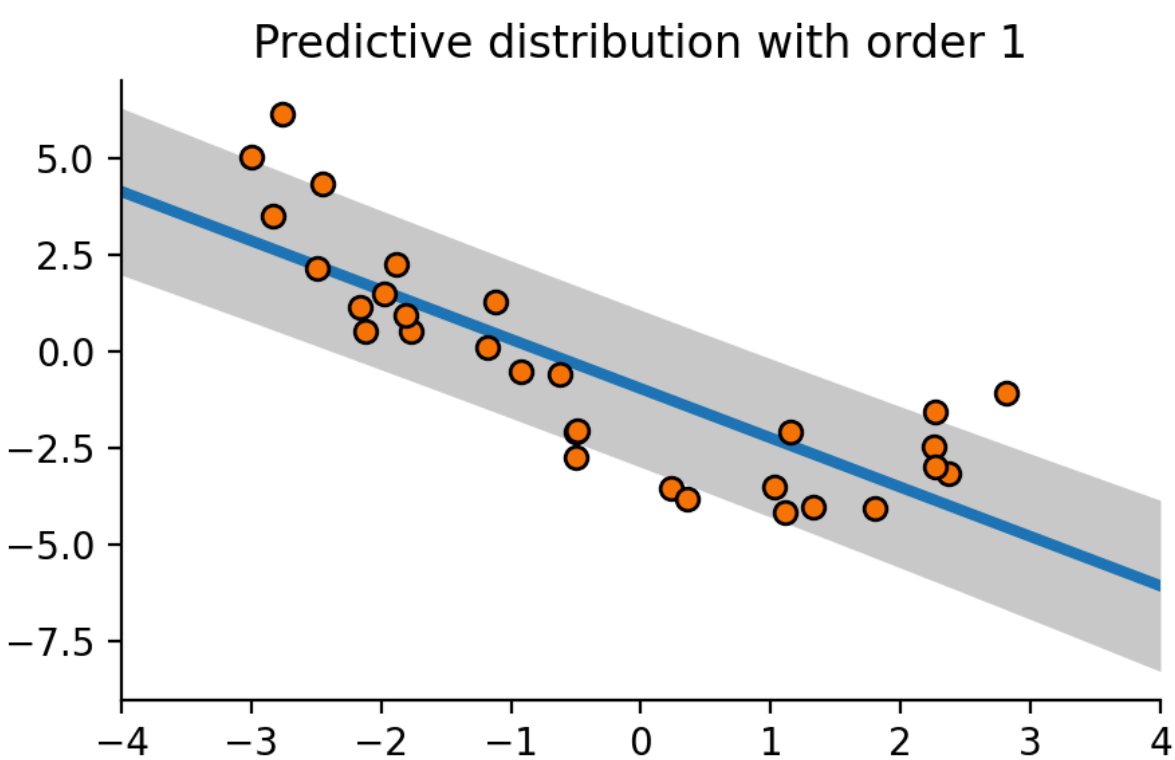
```
def compute_predictive(Xt, w_mean, w_cov, sn2):
    def _compute_predictive_single_point(xt_i, w_mean, w_cov, sn2):
        # @@ COMPLETE @@ #
        yt_i_mean = xt_i @ w_mean
```

```
        yt_i_var = xt_i @ w_cov @ xt_i + sn2
        return yt_i_mean, yt_i_var

    yt_mean, yt_var = np.zeros(len(Xt)), np.zeros(len(Xt))
    for i, xt_i in enumerate(Xt):  # Loop on all the points
        yt_mean[i], yt_var[i] = _compute_predictive_single_point(
            xt_i, w_mean, w_cov, sn2
        )

    return yt_mean, yt_var
```

Exercise: Compute and plot the predictive distribution for 100 points between -4 and +4.

In [157…

```
xt = np.linspace(-4, 4, 250)
Xt = build_features(xt, K)
# @@ COMPLETE @@ #
y_mean, y_var = compute_predictive(Xt, w_posterior_mean, w_posterior_cov, sr

fig, ax = plt.subplots(figsize=[5, 3])
ax.scatter(x, y, edgecolor="black", linewidth=1, facecolor="xkcd:orange", zc
ax.plot(xt, y_mean, color="tab:blue", lw=3)

lb = y_mean - 2 * np.sqrt(y_var)
ub = y_mean + 2 * np.sqrt(y_var)
ax.fill_between(xt, lb, ub, color=".80", lw=0)
ax.set_title(f"Predictive distribution with order {K}")
plt.show()
```

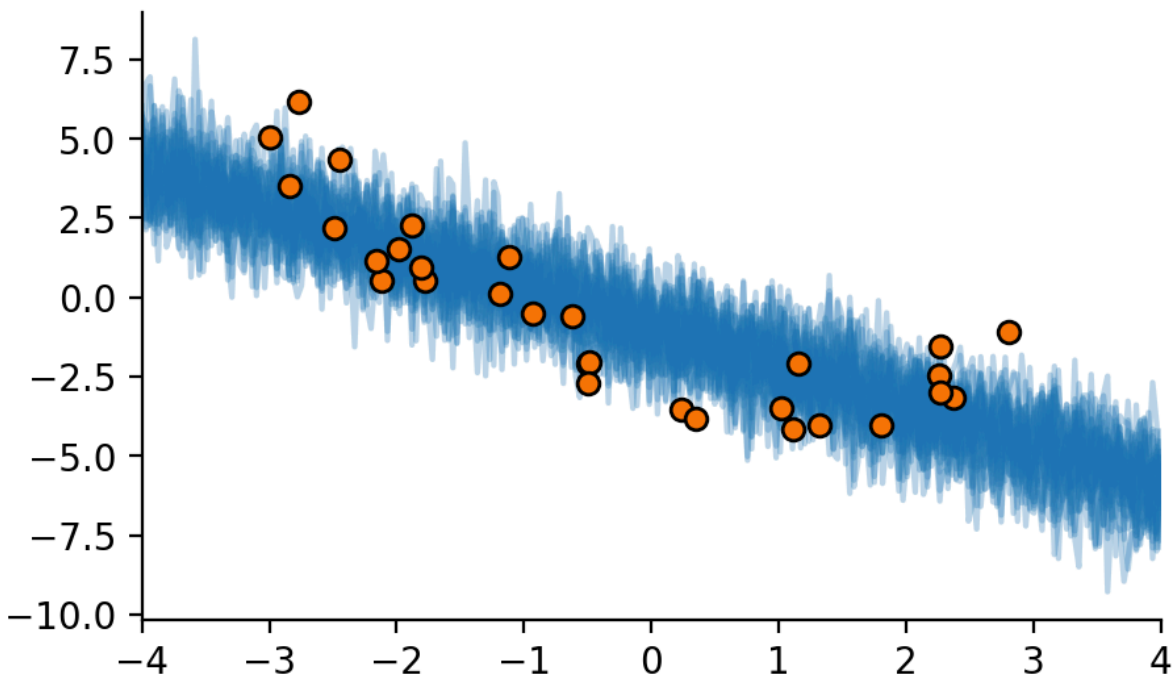


Exercise: You can also sample from the posterior and compute the function values. A simple way to do so is to sample from the posterior on the weights and then from the

noise model (in our case, its $\mathcal{N}(0, \sigma_n^2)$. Complete and use the utility function below: sample 30 times the predictive posterior and plot it.

In [158…

```
def sample(Xt, w_mean, w_cov, sn2, N):
    def _sample_single(Xt, w_mean, w_cov, sn2):
        # @@ COMPLETE @@ #
        w_sample = np.random.multivariate_normal(w_mean, w_cov)
        y_sample = Xt @ w_sample + np.sqrt(sn2) * np.random.randn(len(Xt))
        return y_sample

    samples = np.zeros((N, len(Xt)))
    for i in range(N):
        samples[i] = _sample_single(Xt, w_mean, w_cov, sn2)
    return samples


samples = sample(Xt, w_posterior_mean, w_posterior_cov, sn2, N=30)
fig, ax = plt.subplots(figsize=[5, 3])
ax.scatter(x, y, edgecolor="black", linewidth=1, facecolor="xkcd:orange", zo
ax.plot(xt, samples.T, "tab:blue", alpha=0.3)
plt.show()
```

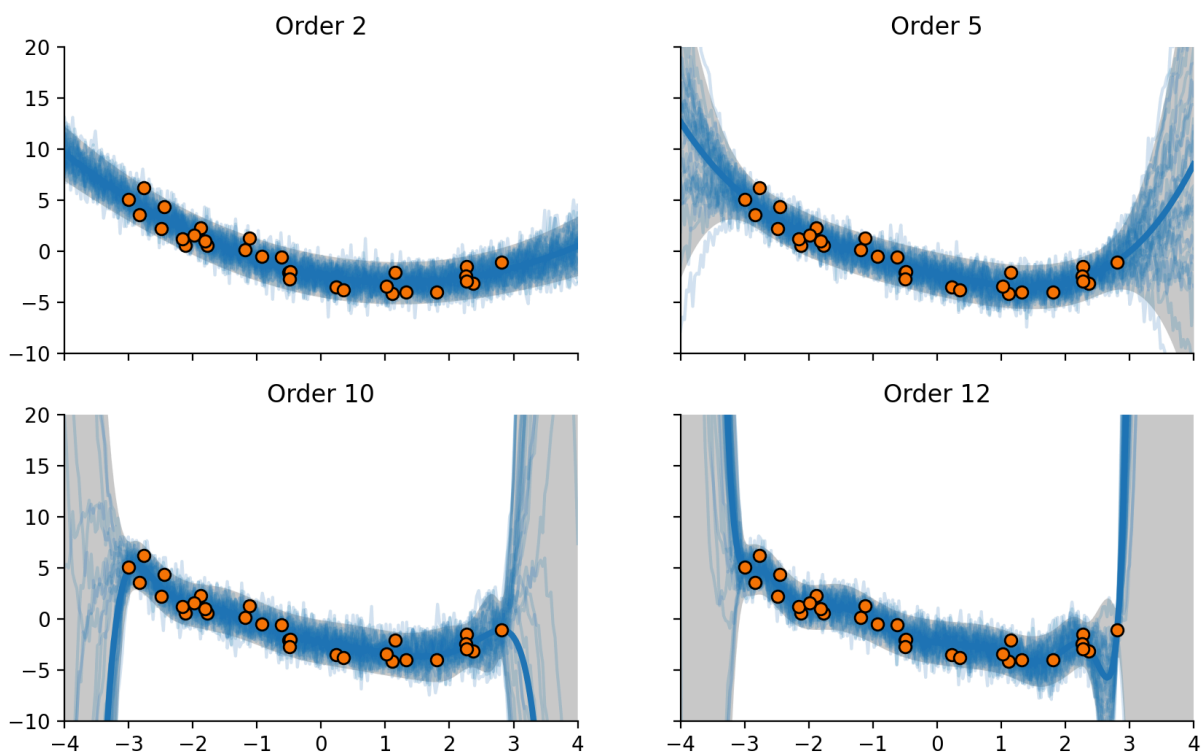


Exercise: Try now with different polynomial order. Let's say 2, 5, 10, 15. Compute the design matrix, the posterior on $\mathbf{w}$ and the predictive posterior. What do you observe?

In [159…

```
def bayesian_linear_regression(x, y, xt, K, sn2, sw2):
    # @@ COMPLETE @@ #
    X = build_features(x, K)
    Xt = build_features(xt, K)
    w_posterior_mean, w_posterior_cov = compute_posterior(X, y, sw2, sn2)
    y_mean, y_var = compute_predictive(Xt, w_posterior_mean, w_posterior_cov
    samples = sample(Xt, w_posterior_mean, w_posterior_cov, sn2, N=30)
```

```
    return y_mean, y_var, samples


poly_orders = [2, 5, 10, 12]

fig, axs = plt.subplots(2, 2, figsize=[10, 6], sharex=True, sharey=True)
axs = axs.reshape(-1)
for ax, K in zip(axs, poly_orders):
    y_mean, y_var, samples = bayesian_linear_regression(x, y, xt, K, sn2, sw

    lb = y_mean - 2 * np.sqrt(y_var)
    ub = y_mean + 2 * np.sqrt(y_var)

    ax.fill_between(xt, lb, ub, color=".80", lw=0)
    ax.plot(xt, samples.T, "tab:blue", alpha=0.2)
    ax.plot(xt, y_mean, color="tab:blue", lw=3)
    ax.scatter(x, y, edgecolor="black", linewidth=1, facecolor="xkcd:orange"
    ax.set_ylim(-10, 20)
    ax.set_title("Order %d" % K)
plt.show()
```



Exercise: Play with the next cell and try to answer the following questions:

1. Set n=0; what is the plot showing?
2. Try to increase the number of points "observed"; what is happening to the posterior?
3. Try to increase the polynomial order; how are the functions behaving?

1. The plot is showing the prior distribution over functions, which is very wide and includes a lot of different functions since we have no data to constrain it.

2. As we increase the number of observed points, the posterior distribution over functions becomes more concentrated around the true underlying function that generated the data. The uncertainty decreases as we have more data to inform our beliefs about the function.
3. As we increase the polynomial order, the functions become more flexible and can fit the data more closely. However, this can also lead to overfitting, where the model captures noise in the data rather than the underlying trend. The functions may become more wiggly and less smooth as the polynomial order increases, especially if the number of observed points is small.

In [160…

```
import ipywidgets as widgets
from ipywidgets import fixed, interact, interact_manual, interactive


def animate(n, K):
    y_mean, y_var, samples = bayesian_linear_regression(x[:n], y[:n], xt, K,
    fig, ax = plt.subplots(figsize=[5, 3])
    ax.scatter(
        x[:n], y[:n], edgecolor="black", linewidth=1, facecolor="xkcd:orange
    )
    ax.plot(xt, samples.T, "tab:blue", alpha=0.3)
    ax.set_ylim(-10, 20)
    plt.show()


interaction = interact(
    animate,
    n=widgets.IntSlider(min=0, max=len(x), step=1, value=0, continuous_updat
    K=widgets.IntSlider(min=0, max=10, step=1, value=1, continuous_update=Fa
)
```

```
interactive(children=(IntSlider(value=0, continuous_update=False, descriptio
n='n', max=30), IntSlider(value=1,…
```

5. Evaluate you model: the marginal likelihood

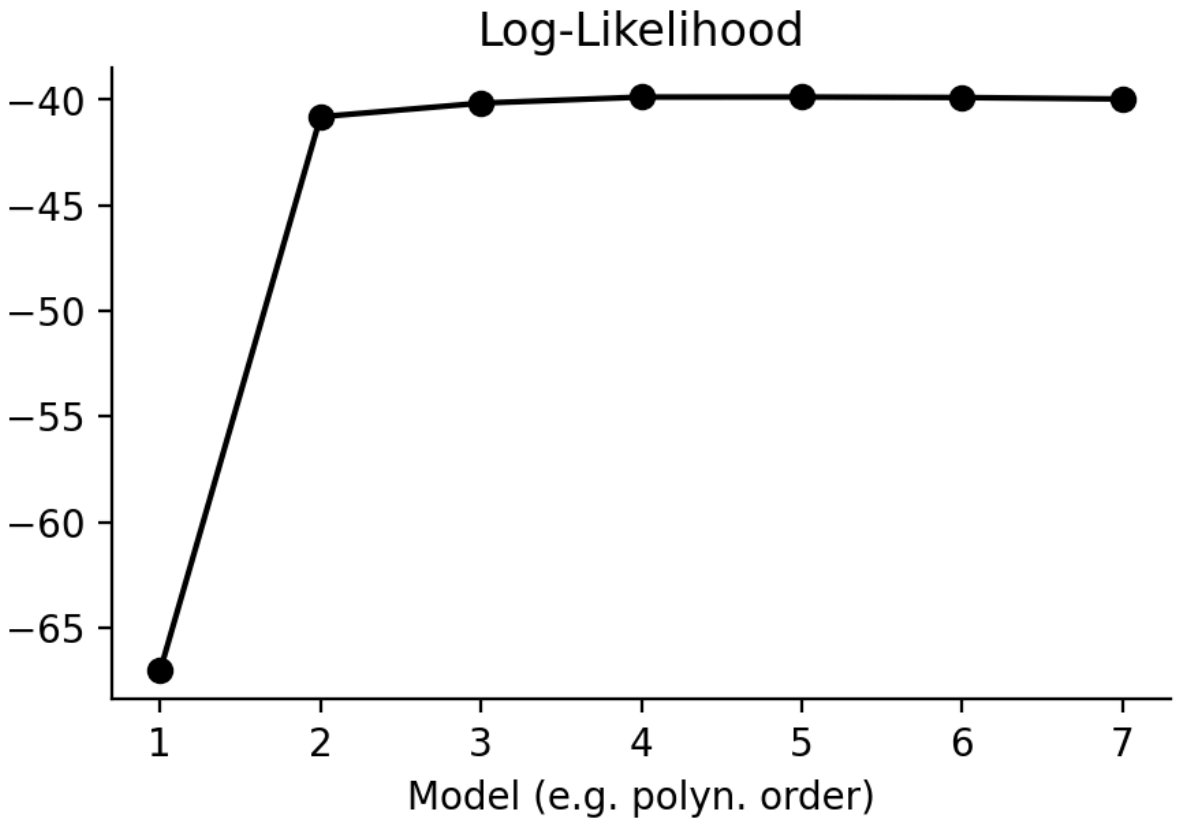
There are several ways in which you can compute the goodness of your model. The first is the likelihood itself.

Question: Compute the loglikelihood for model with order from 0 to 7 and plot it. Comment the results.

In [161…

```
def evaluate_likelihood(x, y, K, sn2, sw2):
    # @@ COMPLETE @@ #
    y_mean, y_var, _ = bayesian_linear_regression(x, y, x, K, sn2, sw2)
    ll = np.sum(gaussian_loglik(y, y_mean, sn2))
    return ll


Ks = np.arange(1, 8)
mll = [evaluate_likelihood(x, y, K, sn2, sw2) for K in Ks]
```

```
fig, ax = plt.subplots(figsize=[5, 3])

ax.plot(Ks, mll, "-ok")

ax.margins(0.05)
ax.set_title("Log-Likelihood")
ax.set_xlabel("Model (e.g. polyn. order)")

plt.show()
```



Question: Try to answer: How likely is $\mathbf{y}$ given $\mathbf{X}$ *and* the model ('first/second/... order polynomial')? Is it the same likelihood as before?

So far, we've ignored $p(\mathbf{y}|\mathbf{X}, \sigma_\mathrm{n}^2)$, the normalising thing in Bayes rule. Being a normalization constant, it has to be equal to

$$p(\mathbf{y}|\mathbf{X}, \sigma_\mathrm{n}^2) = \int p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \sigma_\mathrm{n}^2) p(\mathbf{w}) \mathrm{d}\mathbf{w} \tag{6}$$

We're averaging over all values of $\mathbf{w}$ to get a value for how good the model is.

Question: Suppose the prior being $\mathcal{N}(\mu_0, \mathbf{\Sigma}_0)$ and the likelihood $\mathcal{N}(\mathbf{X}\mathbf{w}, \sigma_\mathrm{n}^2\mathbf{I})$. Derive the marginal likelihood (hint: don't solve the integral -- check the rules for Gaussian conditioning and marginalization) (big hint: check the lecture notes).

Exercise: Write a function to compute the marginal likelihood. Remember: this is a *likelihood* not a density. You should return a number not a density. For simplicity, assume

$\mu_0 = 0$ and $\Sigma_0 = \sigma_{\mathrm{w}}^2 \mathbf{I}$. Use `scipy.stats.multivariate_normal` for computing the logpdf.

From the log-likelihood plot, I see that higher-order models usually fit the training set better. However, this is not always the case, especially when the number of observed points is small. In some cases, higher-order models may overfit the data and have a lower likelihood than simpler models. Therefore, it's important to consider other factors such as model complexity and generalization performance when selecting a model.

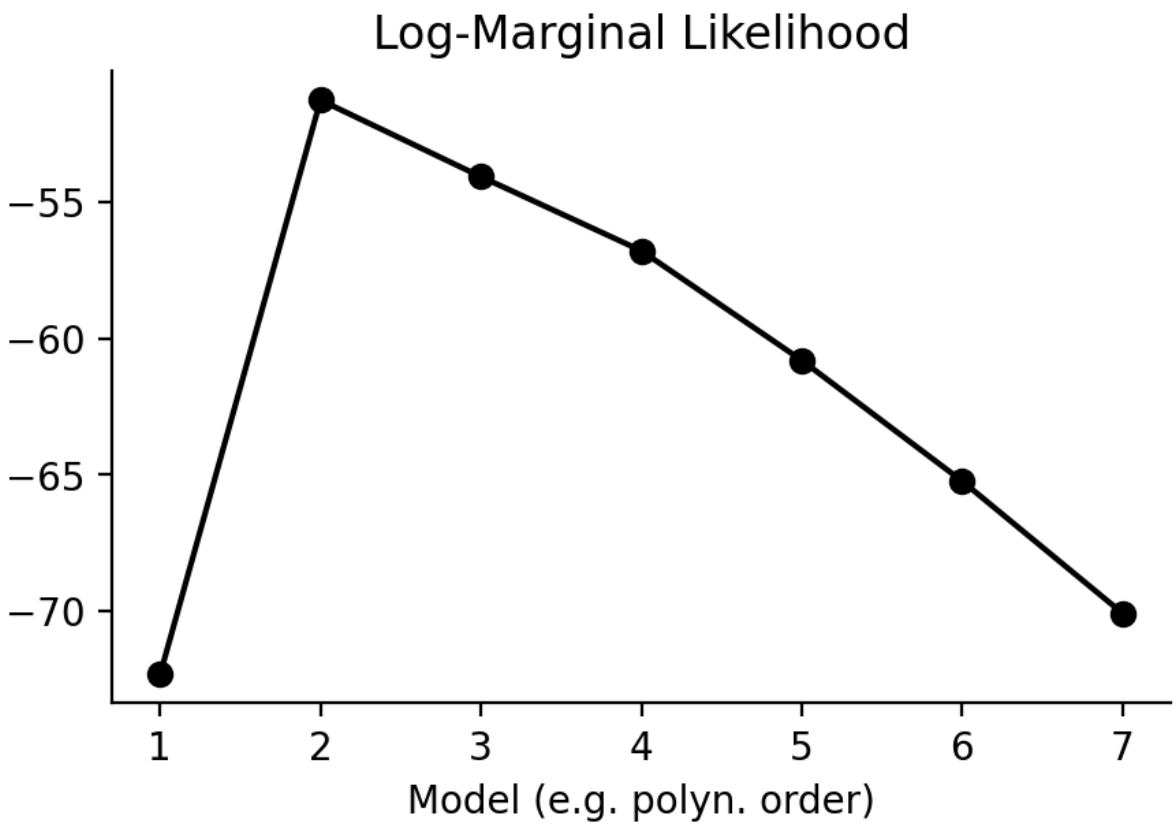
In [162…

```
from scipy.stats import multivariate_normal


def marginal_likelihood(X, y, sw2, sn2):
    # @@ COMPLETE @@ #
    marginal_ll = multivariate_normal.logpdf(y, mean=np.zeros(len(y)), cov=s
    return marginal_ll
```

Exercise: Do the sample plot as before, but now plot the marginal likelihood. You should see a clear pattern here; comment the result.

In [163…

```
def evaluate_marginal_likelihood(x, y, K, sn2, sw2):
    # @@ COMPLETE @@ #
    X = build_features(x, K)
    marginal_ll = marginal_likelihood(X, y, sw2, sn2)
    return marginal_ll


poly_orders = range(1, 8)
mll = [evaluate_marginal_likelihood(x, y, K, sn2, sw2) for K in poly_orders]

fig, ax = plt.subplots(figsize=[5, 3])
ax.plot(poly_orders, mll, "-ok")
ax.margins(0.05)

ax.set_title("Log-Marginal Likelihood")
ax.set_xlabel("Model (e.g. polyn. order)")
plt.show()
```



This Log-marginal likelihood plot should show a clear peak at the optimal polynomial order, which balances model fit and complexity. Models with too low order may underfit the data, while models with too high order may overfit, leading to lower marginal likelihood values. The order of 2 have the highest marginal likelihood, suggesting that it is the best model among the ones considered.

6. When polynomial features are not enough

Run the following cells to download and load a dataset a bit more complex.

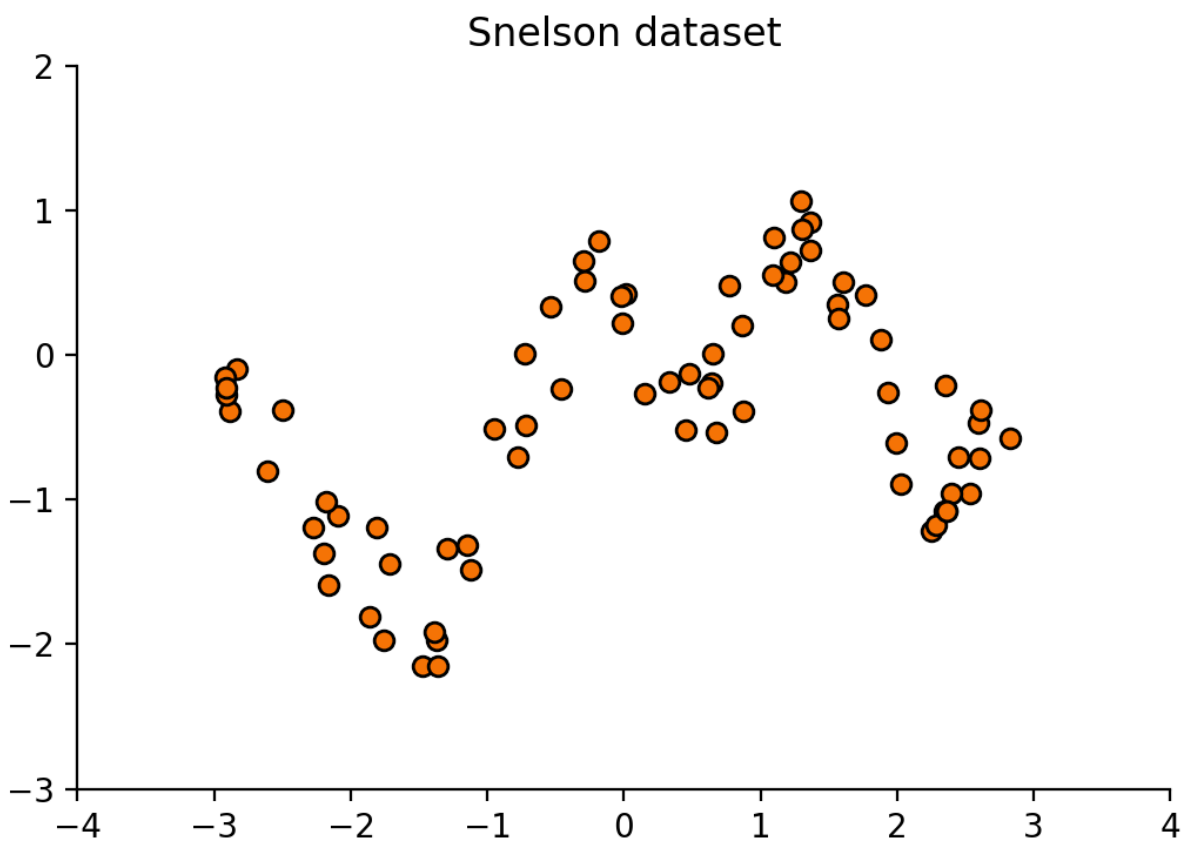
In [164…

```
import os
import urllib.request

from scipy.io import loadmat


def snelson1d(path):
    filename = os.path.join(os.path.expanduser(path), "snelson1d.mat")
    url = "https://github.com/markvdw/gp_upper/raw/master/notebooks/snelson1

    if not os.path.exists(filename):
        print("Downloading from", url)
        urllib.request.urlretrieve(url, filename)

    data = loadmat(filename)
    idx = np.random.RandomState(0).permutation(len(data["X"]))[:75]
```

```
    return data["X"][idx, 0] - 3, data["Y"][idx, 0]


x, y = snelson1d("/tmp/")
xt = np.linspace(-6, 6, 200)
sn2 = 0.1
```

In [165…

```
fig, ax = plt.subplots(figsize=[6, 4])
ax.scatter(x, y, edgecolor="black", linewidth=1, facecolor="xkcd:orange", zo
ax.set_title("Snelson dataset")
ax.set_xlim(-4, 4)
ax.set_ylim(-3, 2)
plt.show()
```



Just from looking at the dataset, you can imagine that fitting this dataset will be hard. And it is (with polynomial features). Maybe it becomes easier with different type of features. Remember that you can choose whatever you want to build the design matrix. Take a look at the following

$$\mathbf{X} = \sqrt{\frac{\alpha^2}{K}} \begin{bmatrix} \cos(\omega_1 x_1) & \sin(\omega_1 x_1) & \dots & \cos(\omega_K x_1) & \sin(\omega_K x_1) \\ \cos(\omega_1 x_2) & \sin(\omega_1 x_2) & \dots & \cos(\omega_K x_2) & \sin(\omega_K x_2) \\ \vdots & \vdots & & \vdots & \\ \cos(\omega_1 x_N) & \sin(\omega_1 x_N) & \dots & \cos(\omega_K x_N) & \sin(\omega_K x_N) \end{bmatrix} \quad \text{where} \quad \omega_i$$

Now, the next formulation of the design matrix $\mathbf{X}$ might seems to come completely out of the blue, but it's not (for those of you interested, this is the random feature expansion of the RBF kernel -- join the next lecture to know more).

For simplicity below you have the code to compute $\mathbf{X}$.

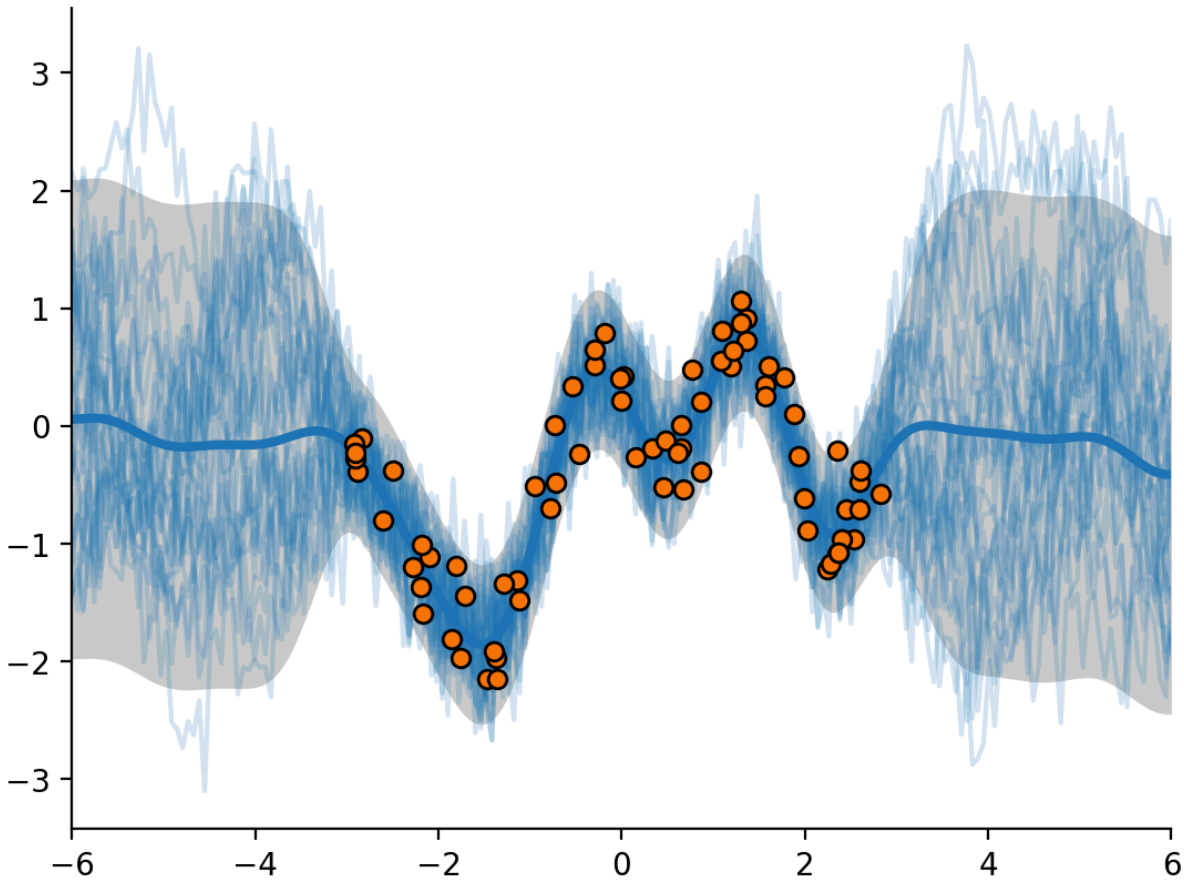
Exercise: Now run Bayesian linear regression on this dataset with this new set of features (NB. You might need many MANY features)

In [166…

```
def build_fourier_features(X: np.ndarray, K: int):
    omega = np.random.RandomState(0).normal(size=K) / 0.5
    sinX_ = np.column_stack([np.sin(X * omega[k]) for k in range(K)])
    cosX_ = np.column_stack([np.cos(X * omega[k]) for k in range(K)])
    return np.concatenate([sinX_, cosX_], -1) * np.sqrt(1 / K)
```

In [167…

```
def bayesian_linear_regression(x, y, xt, K, sn2, sw2):
    # @@ COMPLETE @@ #
    X = build_fourier_features(x, K)
    Xt = build_fourier_features(xt, K)
    w_posterior_mean, w_posterior_cov = compute_posterior(X, y, sw2, sn2)
    y_mean, y_var = compute_predictive(Xt, w_posterior_mean, w_posterior_cov
    samples = sample(Xt, w_posterior_mean, w_posterior_cov, sn2, N=30)
    return y_mean, y_var, samples


fig, ax = plt.subplots()

y_mean, y_var, samples = bayesian_linear_regression(x, y, xt, 100, sn2, sw2)

lb = y_mean - 2 * np.sqrt(y_var)
ub = y_mean + 2 * np.sqrt(y_var)

ax.fill_between(xt, lb, ub, color=".80", lw=0)
ax.plot(xt, samples.T, "tab:blue", alpha=0.2)
ax.plot(xt, y_mean, color="tab:blue", lw=3)
ax.scatter(x, y, edgecolor="black", linewidth=1, facecolor="xkcd:orange", zo
plt.show()
```



Use the cell below to play with the number of features in the design matrix with Fourier features.

In [168…

```
import ipywidgets as widgets
from ipywidgets import fixed, interact, interact_manual, interactive


def animate_random_features(K):
    fig, ax = plt.subplots()

    y_mean, y_var, samples = bayesian_linear_regression(x, y, xt, K, sn2, sw

    lb = y_mean - 2 * np.sqrt(y_var)
    ub = y_mean + 2 * np.sqrt(y_var)

    ax.fill_between(xt, lb, ub, color=".80", lw=0)
    ax.plot(xt, samples.T, "tab:blue", alpha=0.2)
    ax.plot(xt, y_mean, color="tab:blue", lw=3)
    ax.scatter(x, y, edgecolor="black", linewidth=1, facecolor="xkcd:orange"
    ax.set_ylim(-3, 3)
    plt.show()


interaction = interact(
    animate_random_features,
    K=widgets.IntSlider(min=1, max=500, step=10, value=10, continuous_update
)
```

```
interactive(children=(IntSlider(value=10, continuous_update=False, descripti
on='K', max=500, min=1, step=10), …
```

7. A more complex model: being Bayesian on the noise

Now we are ready spice things up! Everything we did, we did it assuming that the noise variance σ_{n}^2 was known. This is a far too restrictive assumption in practice. There are several ways to choose σ_{n}^2 (cross-validation, maximization of the marginal likelihood w.r.t. σ_{n}^2, ...). But this -- being a course on Bayesian inference -- requires to develop a solution in a "Bayesian" way: place a prior on σ_{n}^2 and infer a posterior given some data.

As usual, the likelihood has the form $p(\mathbf{y}|\mathbf{w}, \mathbf{X}, \sigma_{\mathrm{n}}) = \mathcal{N}(\mathbf{X}\mathbf{w}, \sigma_{\mathrm{n}}^2\mathbf{I})$.

One can show that the natural conjugate prior is a normal-inverse Gaussian distribution with the following form:

$$p(\mathbf{w}, \sigma_{\mathrm{n}}^2) = \mathrm{NIG}(\mathbf{w}, \sigma_{\mathrm{n}}^2|\mu_0, \mathbf{\Sigma}_0, a_0, b_0) =$$

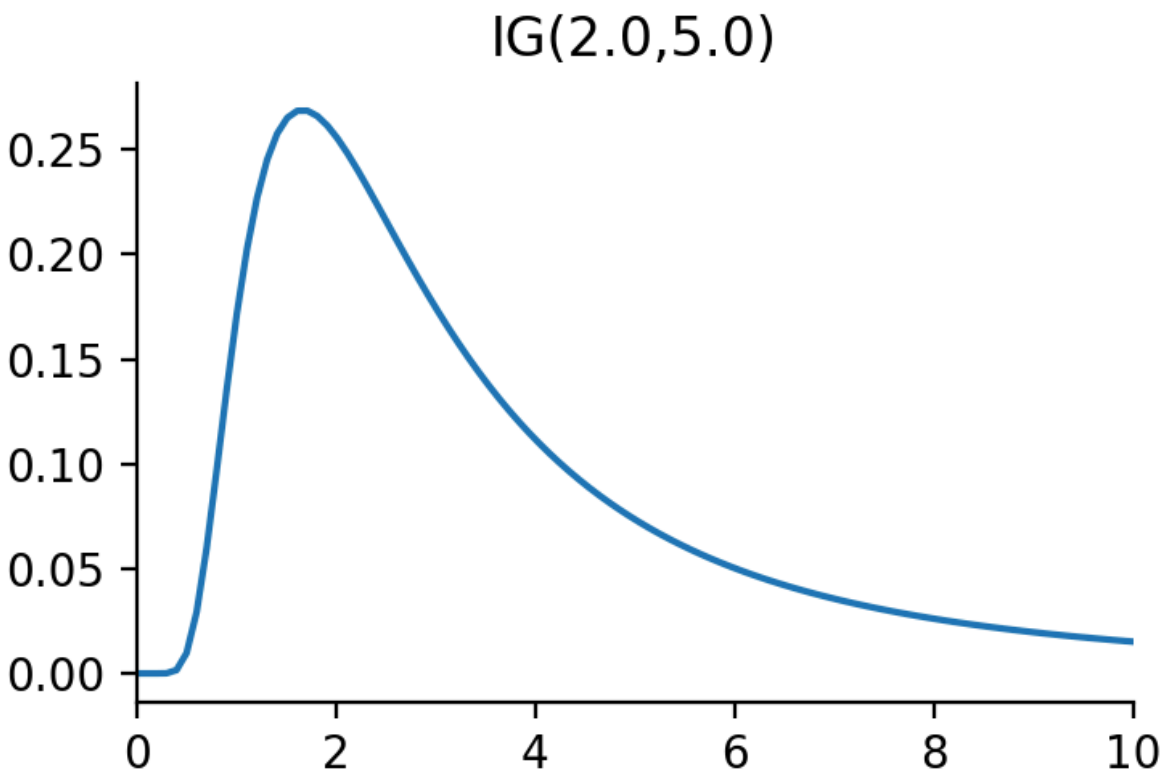
$$= \mathcal{N}(\mathbf{w}|\mu_0, \sigma_{\mathrm{n}}^2\mathbf{\Sigma}_0)\mathrm{IG}(\sigma_{\mathrm{n}}^2|a_0, b_0)$$

where IG is the inverse Gamma distribution.

Exercise: Since changes are you never saw an inverse Gamma distribution, use the following cell to play with it. Try to change the parameters a_0 and b_0. What happens if you both set them to 0?

In [169…

```
a0, b0 = 2, 5
rv = scipy.stats.invgamma(a=a0, scale=b0)

fig, ax = plt.subplots(figsize=[4, 2.5])
xplot = np.linspace(0, 10, 100)
ax.plot(xplot, rv.pdf(xplot))
ax.set_title("IG(%.1f,%.1f)" % (a0, b0))
plt.show()
```



One can derive the posterior $p(\mathbf{w}, \sigma_{\mathrm{n}}^2|\mathbf{X}, \mathbf{y})$ as follows

$$
\begin{aligned}
p(\mathbf{w}, \sigma_{\mathrm{n}}^2|\mathbf{X}, \mathbf{y}) &= \mathrm{NIG}(\mathbf{w}, \sigma_{\mathrm{n}}^2|\mu, \mathbf{\Sigma}, a, b) && (7)\\
\mathbf{\Sigma} &= (\mathbf{\Sigma}_0^{-1} + \mathbf{X}^\top \mathbf{X})^{-1} && (8)\\
\mu &= \mathbf{\Sigma}(\mathbf{\Sigma}_0^{-1}\mu_0 + \mathbf{X}^\top \mathbf{y}) && (9)\\
a &= a_0 + n/2 && (10)\\
b &= b_0 + \frac{1}{2}(\mu_0^\top \mathbf{\Sigma}_0 \mu_0 + \mathbf{y}^\top \mathbf{y} - \mu^\top \mathbf{\Sigma} \mu) && (11)
\end{aligned}
$$

This is left as an exercise to the reader (joking aside if you want to know more check Sec. 7.6.3.1 of "Machine Learning: A probabilistic perspective" by K.P. Murphy). From this formulation, you can derive the two marginals on $\mathbf{w}$ and σ_{n}^2, which are easier to understand.

$$
\begin{aligned}
p(\sigma_{\mathrm{n}}^2|\mathrm{X}, \mathrm{y}) &= \mathrm{IG}(a, b) && (12)\\
p(\mathbf{w}|\mathrm{X}, \mathrm{y}) &= \mathcal{T}\left(\mu, \frac{b}{a}\Sigma, 2a\right) && (13)
\end{aligned}
$$

where $\mathcal{T}$ is a student-T distribution.

Finally, the predictive distribution a new test points $\mathbf{X}_{\mathrm{new}}$ is again a student-T distribution with the following form,

$$
p(\mathbf{y}_{\mathrm{new}}|\mathbf{X}_{\mathrm{new}}, \mathbf{X}, \mathbf{y}) = \mathcal{T}\left(\mathbf{X}_{\mathrm{new}}\mu, \frac{b}{a}(I + \mathbf{X}_{\mathrm{new}}\mathbf{\Sigma}\mathbf{X}_{\mathrm{new}}^\top), 2a\right) \qquad (14)
$$

Exercise: Write the function to compute the posterior following the formula above. Compute also the predictive posterior and plot few samples from it.

For the prior, choose $\mu_0 = 0$, $\mathbf{\Sigma}_0 = \mathbf{I}$ and $a_0 = b_0 = 0$.

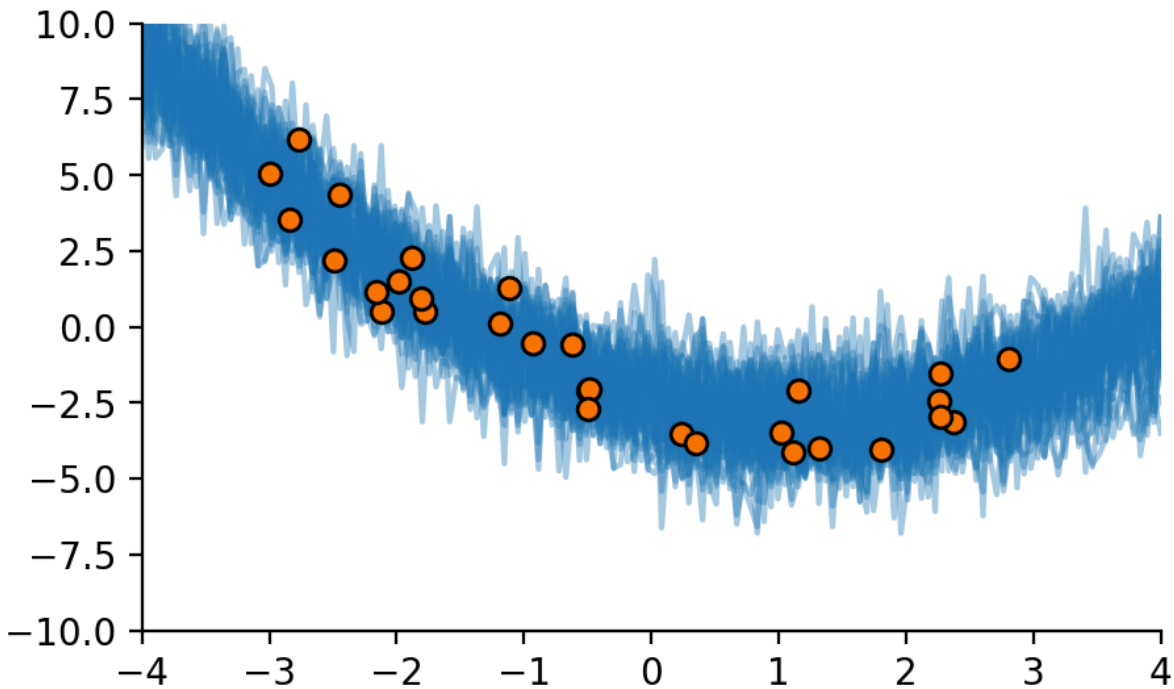
In [170…

```
def compute_posterior(X, y, mu0, Sigma0, a0, b0):
    # @@ COMPLETE @@ #
    Sigma0_inv = np.linalg.inv(Sigma0)
    Sigma_inv = Sigma0_inv + X.T @ X
    Sigma = np.linalg.inv(Sigma_inv)
    mu = Sigma @ (Sigma0_inv @ mu0 + X.T @ y)
    a = a0 + len(y) / 2
    b = b0 + 0.5 * (y.T @ y + mu0.T @ Sigma0_inv @ mu0 - mu.T @ Sigma_inv @
    return mu, Sigma, a, b


def compute_predictive_posterior(X, mu, Sigma, a, b):
    # @@ COMPLETE @@ #
    m = X @ mu
    c = X @ Sigma @ X.T + b / (a - 1) * np.eye(len(X))
    dof = 2 * a
    return m, c, dof


def sample_from_student_t(m, S, dof, n=1):
    d = len(m)
    if dof == np.inf:
        x = 1.0
    else:
        x = np.random.chisquare(dof, n) / dof
    z = np.random.multivariate_normal(np.zeros(d), S, (n,))
    return m + z / np.sqrt(x)[:, None]


def sample(Xt, w_mean, w_cov, w_dof, a, b, N):
    def _sample_single(Xt, w_mean, w_cov, w_dof, a, b):
        # @@ COMPLETE @@ #
        y_sample = sample_from_student_t(Xt @ w_mean, Xt @ w_cov @ Xt.T + b
        return y_sample

    samples = np.zeros((N, len(Xt)))
    for i in range(N):
        samples[i] = _sample_single(Xt, w_mean, w_cov, w_dof, a, b)
    return samples
```

In [176…

```
np.random.seed(1)
sn2 = 1.5
x, y = make_regression(30, sn2=sn2)
xt = np.linspace(-4, 4, 150)

K = 2
X = build_features(x, K)
Xt = build_features(xt, K)
mu0 = np.zeros(X.shape[1])
Sigma0 = np.eye(X.shape[1])
```

```
a0, b0 = 0, 0

mu, Sigma, a, b = compute_posterior(X, y, mu0, Sigma0, a0, b0)

samples = sample(Xt, mu, b / a * Sigma, 2 * a, a, b, 50)
fig, ax = plt.subplots(figsize=[5, 3])
ax.plot(xt, samples.T, "tab:blue", alpha=0.4)
ax.scatter(x, y, edgecolor="black", linewidth=1, facecolor="xkcd:orange", zo
ax.set_ylim(-10, 10)
plt.show()
```



Question: Compare this plot with the previous one with order 1. Do you see any difference? Comment the results.

Exercise: Do the same a before, but now with order 2, 3, 5 and 10.

The plot looks similar to the previous one, and the order of 2 seems to be the best one. However, the predictive distribution is now a student-T distribution, which has heavier tails than a Gaussian distribution. This means that the model is more robust to outliers in the data, and the uncertainty in the predictions is higher compared to the previous case where we assumed a known noise variance.